3

BÖLÜM: KONTEYNER DÜNYASI & DOCKER TEMELLERI | MODULE: CONTAINER WORLD & DOCKER FUNDAMENTALS

Java Spring Boot Uygulamasını Paketleme: Multi-Stage Build & Layered JAR

Packaging Java Spring Boot: Multi-Stage Builds & Layered JARs

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kocaman un çuvallarını, mikseri ve kirli tencereleri salondaki yemek masasına koymazsınız! Mutfakta yemeği pişirir (Maven ile derleme), salona garsonla sadece şık porselen tabaktaki yemeği yollarsınız (JRE). Multi-stage build tam olarak budur: Derleme araçlarını çöpe at, sadece çalışan Java kodunu son imaja koy!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't serve your dinner guests by wheeling 50kg sacks of flour and dirty mixing bowls into the dining room! You cook in the kitchen (Maven build) and carry only the clean porcelain plate to the table (JRE). Multi-stage builds discard the 800MB compiler tools and package only the lean 50MB runtime!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Spring Boot 2.3+ ile gelen 'Layered JAR' mimarisi, geleneksel tek parça (fat/uber) JAR'ları 4 mantıksal katmana böler: 1) dependencies (nadiren değişir), 2) spring-boot-loader, 3) snapshot-dependencies, 4) application (sizin kodunuz; her commit'te değişir). Dockerfile'da `java -Djarmode=layertools -jar app.jar extract` çalıştırarak bu katmanları ayırırız. Böylece 1 satır Java kodu değiştirdiğinizde Docker sadece 1MB'lık application katmanını yeniden derler; 150MB'lık kütüphaneleri baştan indirmez!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Spring Boot layered JARs separate fat JARs into 4 distinct layers: 1) dependencies, 2) spring-boot-loader, 3) snapshot-dependencies, 4) application code. By running `java -Djarmode=layertools -jar app.jar extract`, Docker caches the 150MB third-party libraries indefinitely. When you change one Java class, Docker only invalidates and pushes the 2MB application layer!

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Dockerfile'da Katmanları Çıkarma ve Kopyalama:
FROM eclipse-temurin:21-jre-alpine AS extractor
WORKDIR /build
COPY target/spring-boot-k8s-eli5-1.0.0.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract

FROM eclipse-temurin:21-jre-alpine AS runner
WORKDIR /app
COPY --from=extractor /build/dependencies/ ./
COPY --from=extractor /build/spring-boot-loader/ ./
COPY --from=extractor /build/snapshot-dependencies/ ./
COPY --from=extractor /build/application/ ./
ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Multi-stage kullanmazsanız, JDK derleyicisi ve Maven yerel deposu (/.m2) imajın içinde kalır; imaj boyutunuz 250MB yerine 1.2GB olur ve güvenlik açıkları tavan yapar!

🇬🇧 WATCH OUT

Without multi-stage builds, your JDK compiler, git, and /.m2 cache leak into production, inflating your image from 200MB to 1.5GB with huge security risks.

🎯 Özet Çıkarımlar

- Multi-stage build ile derleme ortamı ve çalışma ortamı tamamen ayrılır.
- Spring Boot Layered JAR ile Docker derleme süreleri %90 hızlanır.
- Konteyner imajı küçüldükçe ağ transferi hızlanır, güvenlik açıkları azalır.

🎯 Key Takeaways

- Multi-stage builds decouple build tools from the minimal runtime image.
- Layered JAR caching reduces image build and push times by up to 90%.
- Smaller images transfer faster and significantly reduce attack surface.